PERCONA®

PERCONA
for PostgreSQL

Transparent Data Encryption (TDE)

©2025 Percona

Overview

- whoami
- whoareu
- purpose of this presentation
- breakdown

©2025 Percona

Database Encryption, Implementations

Types

- Host Based Authentication (pg_hba.conf)
- Secure Socket Layer (SSL)
  - encrypted sessions
  - variable encryption cyphers
- encrypted data
  - pg_crypto
  - translucent data
- encryption at rest
  - filesystem
  - ***transparent data encryption***
    - ***Relations***
    - ***WAL***

©2025 Percona

ISO 27001:2022 Annex A 8.24

the use of cryptography at rest (TDE)

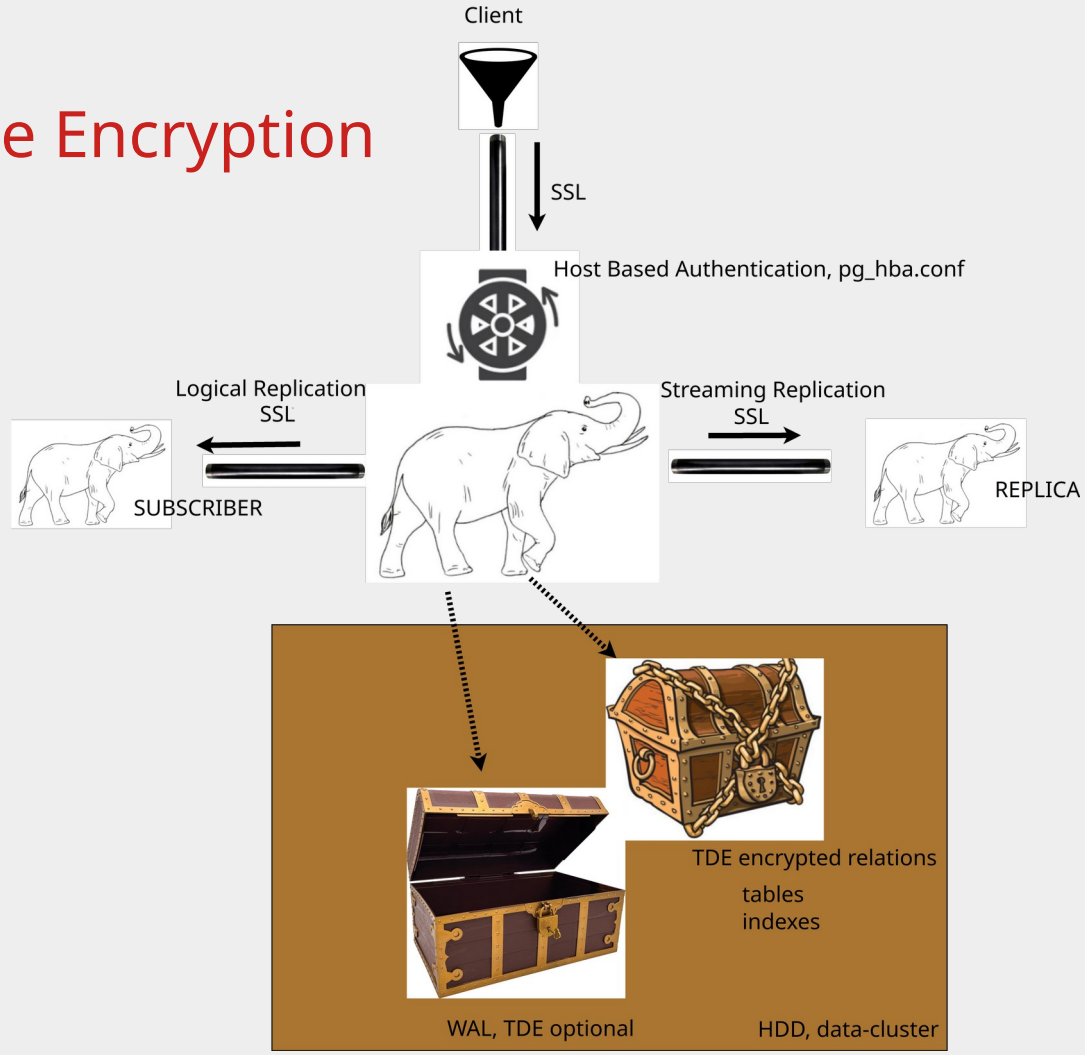
©2025 Percona

pg_tde, a closer look … What’s Encrypted?

- Data is encrypted on disk
- Indexes, Sequences encrypted on disk
- WALS encrypted on disk
- Data can be queried
- Can perform DML and DDL operations

©2025 Percona

Total Database Encryption



©2025 Percona

Performance Objective

Develop and implement fully encrypted relations into a production environment.

©2025 Percona

Enabling Objective

Overview

- Part I: Setup
- Part II: Working with KeyRings
- Part III:Enabling WAL encryption
- Part IV: Working with Replication
- Part V: Administering OpenBao

©2025 Percona

Installing Percona PostgreSQL

```
apt-get update && apt-get upgrade -y

# needed to install Percona PostgreSQL
apt-get install -y curl wget gnupg2
```

```
# Percona PostgreSQL, includes the pg_tde extension
wget https://repo.percona.com/apt/percona-release_latest.generic_all.deb
dpkg -i ./percona-release_latest.generic_all.deb

##
## Reference:
##  percona-release | less
##

# enable repository
percona-release enable ppg-17.5 release

apt-get install -y percona-postgresql-17
```

```
pg_dropcluster 17 main --stop
echo postgres > /tmp/a
pg_createcluster 17 main -- --pwfile=/tmp/a
rm -f /tmp/a

pg_conftool 17 main postgresql.auto.conf set shared_preload_libraries 'pg_tde'
pg_conftool 17 main postgresql.auto.conf set wal_level 'logical'
```


©2025 Percona

Configuring: Create a Template Database

```
# as postgres
sudo su - postgres <<_eof_
    createdb tde_template
    psql tde_template <<_eof1_
        create extension pg_tde;
        select * from pg_am;
_eof1_
_eof_
```

```
  oid   |  amname  |       amhandler        | amtype
-------+----------+------------------------+--------
     2 | heap     | heap_tableam_handler   | t
   403 | btree    | bthandler              | i
   405 | hash     | hashhandler            | i
   783 | gist     | gisthandler            | i
  2742 | gin      | ginhandler             | i
  4000 | spgist   | spghandler             | i
  3580 | brin     | brinhandler            | i
 16431 | tde_heap | pg_tdeam_handler       | t
(8 rows)
```


©2025 Percona

pg_tde: table

```
                                    Table "public.t1"
  Column  |           Type           | Collation | Nullable |            Default
----------+--------------------------+-----------+----------+--------------------------------
 id       | integer                  |           | not null | nextval('t1_id_seq'::regclass)
 comments | text                     |           |          | 'hello world'::text
 t_stamp  | timestamp with time zone |           |          | now()
Indexes:
    "t1_pkey" PRIMARY KEY, btree (id)
Access method: tde_heap
```

```
db01=# select datname, datid from pg_stat_database where datname='db01';
 datname | datid
---------+-------
 db01    | 16450

db01=# select relname,relfilenode from pg_class where relname='t1' or relname='t1_pkey';
 relname | relfilenode
---------+-------------
 t1      |       16462
 t1_pkey |       16467
```


©2025 Percona

pg_tde: datacluster

```
tree /var/lib/postgresql/17/main/base/16450
.
.
.
├── 16462
├── 16462_fsm
├── 16462_v
.
.
├── 16467
.
.
.
```

```
tree /var/lib/postgresql/17/main/
.
.
.
── pg_tde
│   ├── pg_tde_16450_dat
│   ├── pg_tde_16450_map
│   ├── pg_tde_1663_dat
│   ├── pg_tde_1663_map
│   └── pg_tde_607_keyring
.
.
.
├── pg_wal
│   ├── 000000010000000000000001D
│   ├── 000000010000000000000001E
│   ├── 000000010000000000000001F
│   ├── 000000010000000000000020
```


©2025 Percona

Creating the Encrypted Table: Steps

Step 1: Add a key provider

1) keyfile (localhost)
2) Key Management servers (OpenBao)

Step 2: Set the key

Step 3: Create the table/relation

©2025 Percona

Creating the Encrypted Table: Functions

```
db01=# \df pg_tde_add*provider*
                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                List of functions
 Schema |                    Name                    | Result data type |                                                                                                                                                                                                                                                                                                                
```

Creating the Encrypted Table: Functions, cont'd

```
db01=# \df pg_tde_list*
                                                                List of functions
 Schema |                  Name                   | Result data type |                                Argument data types                                |
--------+-----------------------------------------+------------------+-----------------------------------------------------------------------------------+
 public | pg_tde_list_all_database_key_providers | SETOF record     | OUT id integer, OUT provider_name text, OUT provider_type text, OUT options json |
 public | pg_tde_list_all_global_key_providers   | SETOF record     | OUT id integer, OUT provider_name text, OUT provider_type text, OUT options json |
(2 rows)

db01=# \df pg_tde_*info*
                                                                    List of functions
 Schema |          Name           |                                      Result data type                                                                |
--------+-------------------------+-------------------------------------------------------------------------------------------------------------------+-
 public | pg_tde_default_key_info | TABLE(key_name text, key_provider_name text, key_provider_id integer, key_creation_time timestamp with time zone) |
 public | pg_tde_key_info         | TABLE(key_name text, key_provider_name text, key_provider_id integer, key_creation_time timestamp with time zone) |
 public | pg_tde_server_key_info  | TABLE(key_name text, key_provider_name text, key_provider_id integer, key_creation_time timestamp with time zone) |
(3 rows)
```


©2025 Percona

Examples:

- EX 1: set a default provider and key across all databases
- EX 2: Assign a provider and key unique to a specific database
- EX 3: Delete database db03 provider and provide a new one
- EX 4: Rename key
- EX 5: Update Table Access Method
- EX 6: Replication, Streaming
- EX 7: Replication, Logical

©2025 Percona

EX 1: set a default provider and key across all databases

```
psql db01 <<_eof_

    select * from pg_tde_add_global_key_provider_file (
        'provider_global',
        '/var/lib/postgresql/key_global.dat'
    );

    select * from pg_tde_create_key_using_global_key_provider(
        'key_global',
        'provider_global'
    );

    select * from pg_tde_set_default_key_using_global_key_provider (
        'key_global',
        'provider_global'
    );

    -- validation
    select * from pg_tde_list_all_global_key_providers();
    select * from pg_tde_default_key_info();
_eof_
```


©2025 Percona

EX 1: cont'd

```
psql db01 <<_eof_
    create table t1 (
        id serial primary key,
        comments text default 'hello world',
        t_stamp timestamptz default now()
    ) using tde_heap;

    select * from pg_tde_is_encrypted('t1');

    insert into t1 values (default,default,default);
    table t1;

    create database db02 with template tde_template;

\c db02
    -- validation
    select * from pg_tde_list_all_global_key_providers();
    select * from pg_tde_default_key_info();

    create table t1 (
        id serial primary key,
        comments text default 'hello world',
        t_stamp timestamptz default now()
    ) using tde_heap;

    select * from pg_tde_is_encrypted('t1');

    insert into t1 values (default,default,default);
    table t1;
_eof_
```


©2025 Percona

EX 2: Assign a provider and key unique to a specific database

```
postgres@pg2:~$ createdb db03 --template=tde_template

psql db03 <<_eof_
    select * from pg_tde_add_database_key_provider_file(
        'provider_db03',
        '/var/lib/postgresql/key_db03.dat'
    );

    select * from pg_tde_create_key_using_database_key_provider(
        'key_db03',
        'provider_db03'
    );
```

```
    -- validation
    select * from pg_tde_list_all_database_key_providers();
    select * from pg_tde_key_info();

    create table t1 (
        id serial primary key,
        comments text default 'hello world',
        t_stamp timestamptz default now()
    ) using tde_heap;

    select * from pg_tde_is_encrypted('t1');
_eof_
```


©2025 Percona

EX 3: Create a New Key

Recall:

```
db03=# select * from pg_tde_list_all_database_key_providers();
select * from pg_tde_key_info();
 id |      name      | type |                    options
----+----------------+------+-----------------------------------------------
  2 | provider_db03a | file | {"path" : "/var/lib/postgresql/key_db03a.dat"}
(1 row)

 key_name | provider_name  | provider_id |       key_creation_time
----------+----------------+-------------+-------------------------------
 key_db03 | provider_db03a |           2 | 2025-06-25 21:52:11.276517+00
```

Execute:

```
select * from pg_tde_create_key_using_database_key_provider (
    'key_db03a',
    'provider_db03a'
);

select * from pg_tde_set_key_using_database_key_provider(
    'key_db03a',
    'provider_db03a'
);
```

Validate:

```
 key_name  | provider_name  | provider_id |       key_creation_time
-----------+----------------+-------------+-------------------------------
 key_db03a | provider_db03a |           2 | 2025-06-25 22:01:19.199359+00
```


©2025 Percona

EX 4: Delete database provider

Recall:

```
postgres@pg1:~$ ./ex04.Delete-replace-provider
 id |       name       | type |                      options
----+------------------+------+-----------------------------------------------
  1 | provider_db03    | file | {"path" : "/var/lib/postgresql/key_db03.dat"}
  2 | provider_db03a   | file | {"path" : "/var/lib/postgresql/key_db03a.dat"}
(2 rows)

ERROR:  cannot delete provider which is currently in use
 pg_tde_delete_database_key_provider
-------------------------------------

(1 row)
```

```
-- recall: there are two providers
select * from pg_tde_list_all_database_key_providers();

-- fails: provider is active
select * from pg_tde_delete_database_key_provider('provider_db03a');

-- works: this provider is inactive
select * from pg_tde_delete_database_key_provider('provider_db03');
```


©2025 Percona

WAL Encryption

- Step 1: Add global key provider
- Step 2: Set the server principal key
- Step 3: Update runtime parameter pg_tde.wal_encrypt

©2025 Percona

EX 5: WAL encryption

```
psql db01 <<_eof_
    --
    -- recall provider: provider_global
    --
    select * from pg_tde_list_all_global_key_providers();

    select * from pg_tde_create_key_using_global_key_provider (
        'key-wal',
        'provider_global'
    );

    select * from pg_tde_set_server_key_using_global_key_provider (
        'key-wal',
        'provider_global'
    );

    select name,setting from pg_settings where name~'^pg_tde';
    alter system set pg_tde.wal_encrypt = on;
_eof_
```

```
 id |      name       | type |                    options
----+-----------------+------+-----------------------------------------------
 -1 | provider_global | file | {"path" : "/var/lib/postgresql/key_global.dat"}
(1 row)

 pg_tde_create_key_using_global_key_provider
---------------------------------------------

(1 row)

 pg_tde_set_server_key_using_global_key_provider
-------------------------------------------------

(1 row)

              name               | setting
---------------------------------+---------
 pg_tde.enforce_encryption       | off
 pg_tde.inherit_global_providers | on
 pg_tde.wal_encrypt              | off
(3 rows)
```


©2025 Percona

EX 5: WAL encryption cont'd *(after server restart)*

```
                name                | setting
-----------------------------------+---------
 pg_tde.enforce_encryption         | off
 pg_tde.inherit_global_providers   | on
 pg_tde.wal_encrypt                | on
(3 rows)
```

```
psql -c "insert into t1 select generate_series(101,10E6);"
```

```
db01=# table t1_db01;
 id |        comments        |            t_stamp
----+------------------------+-------------------------------
  1 | hello world, t1_db01 | 2025-05-05 18:04:59.109395+00
  2 | hello world, t1_db01 | 2025-05-05 18:04:59.113778+00
  3 | hello world, t1_db01 | 2025-05-05 18:04:59.12062+00
  4 | hello world, t1_db01 | 2025-05-05 22:53:17.799112+00
  5 | hello world, t1_db01 | 2025-05-05 22:53:19.171206+00
  6 | hello world, t1_db01 | 2025-05-05 22:53:19.795662+00
  7 | hello world, t1_db01 | 2025-05-05 22:53:20.762213+00
(7 rows)
```

Note the error message … WAL is encrypted

```
postgres@pg1:~$ ls -al /var/lib/postgresql/17/main/pg_wal/
total 49160
drwx------ 1 postgres postgres      190 May  5 22:49 .
drwx------ 1 postgres postgres      456 May  5 22:49 ..
-rw------- 1 postgres postgres 16777216 May  5 22:55 000000010000000000000012
-rw------- 1 postgres postgres 16777216 May  5 18:10 000000010000000000000013
-rw------- 1 postgres postgres 16777216 May  5 22:49 000000010000000000000014
drwx------ 1 postgres postgres        0 May  5 22:49 archive_status
drwx------ 1 postgres postgres        0 Apr 23 20:21 summaries
postgres@pg1:~$ /usr/lib/postgresql/17/bin/pg_waldump -f -p /var/lib/postgresql/17/main/pg_wal/ 000000010000000000000012
pg_waldump: error: invalid WAL segment size in WAL file "000000010000000000000012" (1575117056 bytes)
pg_waldump: detail: The WAL segment size must be a power of two between 1 MB and 1 GB.
```


©2025 Percona

Streaming Replication, Steps

- Create PRIMARY cluster
  - Standard Configuration Steps
  - Create template database
    - Add Global Provider (ex: file)
    - Set default principle key
  - Create Database, using template database
    - Create TDE enabled table
- Create REPLICA
  - Create basebackup
  - Configure appropriately
  - Use same TDE provider as PRIMARY

©2025 Percona

EX 6: Streaming Replication

```
PRIMARY (port:5433, password:postgres)

# shutdown existing cluster, prevents confusion
systemctl stop postgresql@17-main

# Execute the following either as root or ubuntu
#   initialize the postgres cluster, preload pg_tde
#
sudo su - postgres <<_eof_
    pg_createcluster 17 pg1_primary -p 5433 -- --pwfile=<(echo 'postgres')
    pg_conftool 17 pg1_primary postgresql.auto.conf set listen_addresses '*'
    pg_conftool 17 pg1_primary postgresql.auto.conf set wal_level 'logical'
    pg_conftool 17 pg1_primary postgresql.auto.conf set shared_preload_libraries 'pg_tde'
_eof_

systemctl start postgresql@17-pg1_primary


sudo su - postgres <<_eof_
    rm -f key_streaming.file
    export PGPORT=5433
    createdb db01

# install default global provider key
    psql -1 db01 <<_eof1_
        create extension pg_tde;
        select pg_tde_add_global_key_provider_file ('provider_global_streaming','/var/lib/postgresql/key_streaming.file');
        select pg_tde_create_key_using_global_key_provider('key_global','provider_global_streaming');
        select pg_tde_set_default_key_using_global_key_provider ('key_global','provider_global_streaming');
_eof1_
_eof_
```


©2025 Percona

EX 6: Streaming Replication cont'd …

```
REPLICA, STREAMING (port:5434, password:postgres)

sudo su - postgres <<_eof_
    pg_createcluster 17 pg2_replica_streaming -p 5434
    rm -rf /var/lib/postgresql/17/pg2_replica_streaming/*

    # keeping things simple, no slots are used
    /usr/bin/pg_basebackup  -d "host=localhost port=5433 user=postgres password=postgres" \
                        -D /var/lib/postgresql/17/pg2_replica_streaming \
                        --wal-method=stream \
                        -c fast \
                        -R -P -v
_eof_

systemctl start postgresql@17-pg2_replica_streaming
```

```
pg_lsclusters

root@pg1-pgtde:~# pg_lsclusters
Ver Cluster               Port Status          Owner    Data directory                                Log file
17  main                  5432 down            postgres /var/lib/postgresql/17/main                   /var/log/postgresql/postgresql-17-main.log
17  pg1_primary           5433 online          postgres /var/lib/postgresql/17/pg1_primary            /var/log/postgresql/postgresql-17-pg1_primary.log
17  pg2_replica_streaming 5434 online,recovery postgres /var/lib/postgresql/17/pg2_replica_streaming /var/log/postgresql/postgresql-17-pg2_replica_streaming.log
```


©2025 Percona

EX6: Streaming Replication cont'd …

```
VALIDATE

# session: PRIMARY
sudo su - postgres <<_eof_
    export PGPORT=5433
    psql db01 -c 'select * into t1 from generate_series(1,10) as c1; alter table t1 set access method tde_heap';
_eof_

# session: REPLICA-STREAMING
sudo su - postgres <<_eof_
    export PGPORT=5434
    psql db01 -c "select * from t1;select * from pg_tde_is_encrypted('t1');"
_eof_
```

```
db_stream=# select * from t1;
 c1
----
  1
  2
  3
  4
  5
  6
  7
  8
  9
 10
(10 rows)

db_stream=# select * from pg_tde_is_encrypted('t1');
 pg_tde_is_encrypted
---------------------
 t
(1 row)
```


©2025 Percona

Logical Replication, Steps

- Create PRIMARY cluster

  (*hint: use previously created primary host*)

  - Create Database, using template database
    - Create TDE enabled table & populate
    - PUBLISH table
- Create REPLICA (logical)
  - Configure appropriately
  - Create template database
    - Add Global Provider (ex: file)
    - Set default principle key
  - Create Database, using template database
    - Create TDE enabled table, do NOT populate
    - SUBSCRIBE table

©2025 Percona

EX7: Logical Replication

```
LOGICAL (port:5435, password:postgres)

# pg_dropcluster 17 pg3_replica_logical --stop

sudo su - postgres <<_eof_
    pg_createcluster 17 pg3_replica_logical -p 5435 -- --pwfile=<(echo 'postgres')
    pg_conftool 17 pg3_replica_logical postgresql.auto.conf set listen_addresses '*'
    pg_conftool 17 pg3_replica_logical postgresql.auto.conf set wal_level 'logical'
    pg_conftool 17 pg3_replica_logical postgresql.auto.conf set shared_preload_libraries 'pg_tde'
_eof_

systemctl start postgresql@17-pg3_replica_logical

sudo su - postgres <<_eof_
    export PGPORT=5435
    createdb db01

# install default global provider key
    psql db01 <<_eof1_
        create extension pg_tde;
        select pg_tde_add_global_key_provider_file('provider_global_logical','/var/lib/postgresql/key_logical.file');
        select pg_tde_set_default_key_using_global_key_provider ('key_global','provider_global_logical');
_eof1_
_eof_
```


©2025 Percona

EX7: Logical Replication cont'd …

```
# SESSION: PRIMARY
sudo su - postgres <<_eof_
    export PGPORT=5433
    dropdb --if-exists db_logical
    createdb db_logical

    psql -1 db_logical <<_eof1_
        create extension pg_tde;
        create publication pub1 for all tables;
        create table t1 (c1 int) using tde_heap;
        insert into t1 select * from generate_series(1,10);
        select * from pg_publication_tables;
        select * from pg_tde_is_encrypted('t1');
_eof1_
_eof_
```


©2025 Percona

EX7: Logical Replication cont'd …

```
# SESSION: REPLICA, LOGICAL
sudo su - postgres <<_eof_
    export PGPORT=5435
    dropdb --if-exists db_logical
    createdb --template=db01 db_logical
    psql db_logical -c 'create table t1 (c1 int) using tde_heap;'
_eof_

sudo su - postgres <<_eof_
    export PGPORT=5435

    psql db_logical <<_eof1_
        create subscription sub1
            connection 'host=localhost port=5433 dbname=db_logical user=postgres password=postgres'
            publication pub1;
_eof1_
    sleep 5s
    psql db_logical <<_eof1_
        select * from t1;
        select * from pg_tde_is_encrypted('t1');

        -- demonstrate replication without tde
        alter table t1 set access method heap;
        select * from pg_tde_is_encrypted('t1');
        \c 'port=5433 dbname=db_logical'
        insert into t1 values(1000);
        \c 'port=5435 dbname=db_logical'
        select * from t1;
_eof1_
_eof_
```


©2025 Percona

Encrypted Replication, CAVEAT

- Regarding Streaming Replication:
  - Ensure the key provider and key are accessible to both PRIMARY and REPLICA
  - regarding a keyring file: use the same one for both hosts
  - regarding an external file/URL:
    - URL is the same
    - key must first be generated ala the unencrypted keyring
- Regarding Logical Replication:
  - use SSL session between publisher and subscriber because logical decoding is not encrypted
  - an encrypted table can be encrypted to a subscribed non-encrypted table
  - if subscription tables are encrypted they can use different encryption keys
  - If WAL encryption is enabled: PUBLICATIONS and SUBSCRIPTIONS are independent of each other can use either the same key or a different one.

©2025 Percona

OpenBao

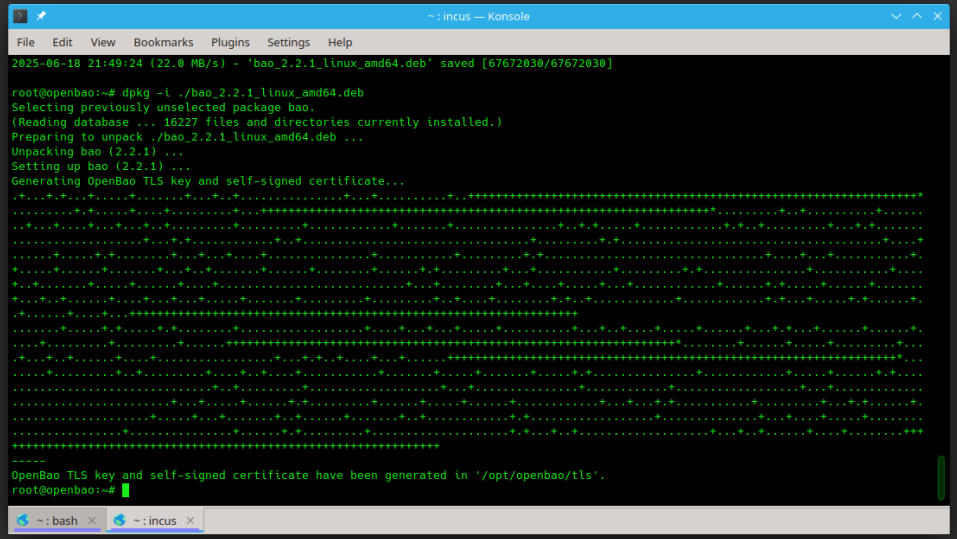
Working With OpenBao

- Installation (Ubuntu 24.04)
- Configuration (basic)
  - certificates
    - without
    - with
  - about the web based user interface i.e. "ui"
- Startup
- Initialization
  - generate an "unseal" key
  - vault token
- Administering with "bao"
  - basic commands
  - environment variables
  - unseal OpenBao
  - create a mount point
- Assign a provider and key to a database

©2025 Percona

Installation

```
# as root
wget https://github.com/openbao/openbao/releases/download/v2.3.0-beta20250528/bao-hsm_2.3.0-beta20250528_linux_amd64.deb
dpkg -i ./bao_2.2.1_linux_amd64.deb
```



©2025 Percona

Configuration

```
cp /etc/openbao/openbao.hcl /etc/openbao/openbao.hcl_DEPRECATED

echo "
ui = true

storage \"file\" {
  path = \"/opt/openbao/data\"
}

# HTTP listener
listener \"tcp\" {
  address = \"0.0.0.0:8200\"
  tls_disable = 1
}
" > /etc/openbao/openbao.hcl

chown openbao:openbao /etc/openbao/openbao.hcl
```


©2025 Percona

Startup & Status

```
root@openbao:~# systemctl status openbao
● openbao.service - "OpenBao - A tool for managing secrets"
     Loaded: loaded (/usr/lib/systemd/system/openbao.service; disabled; preset: enabled)
    Drop-In: /run/systemd/system/service.d
             └zzz-lxc-service.conf
     Active: active (running) since Wed 2025-06-18 21:55:23 UTC; 2min 9s ago
       Docs: https://github.com/openbao/openbao/tree/main/website/content/docs
   Main PID: 1061 (bao)
      Tasks: 17 (limit: 28438)
     Memory: 13.3M (swap max: 0B peak: 15.1M)
        CPU: 148ms
     CGroup: /system.slice/openbao.service
             └1061 /usr/bin/bao server -config=/etc/openbao/openbao.hcl

Jun 18 21:55:23 openbao bao[1061]:                Log Level:
Jun 18 21:55:23 openbao bao[1061]:            Recovery Mode: false
Jun 18 21:55:23 openbao bao[1061]:                  Storage: file
Jun 18 21:55:23 openbao bao[1061]:                  Version: OpenBao v2.2.1, built 2025-04-22T20:36:25Z
Jun 18 21:55:23 openbao bao[1061]:              Version Sha: 91733be3109e5a5000b750939e5748433c78cfcf
Jun 18 21:55:23 openbao bao[1061]: ==> OpenBao server started! Log data will stream in below:
Jun 18 21:55:23 openbao bao[1061]: 2025-06-18T21:55:23.551Z [INFO]  proxy environment: http_proxy="" https_proxy="" no_proxy=""
Jun 18 21:55:23 openbao bao[1061]: 2025-06-18T21:55:23.551Z [WARN]  no `api_addr` value specified in config or in BAO_API_ADDR;
Jun 18 21:55:23 openbao bao[1061]: 2025-06-18T21:55:23.552Z [INFO]  core: Initializing version history cache for core
Jun 18 21:55:23 openbao systemd[1]: Started openbao.service - "OpenBao - A tool for managing secrets".
lines 1-23/23 (END)
```


©2025 Percona

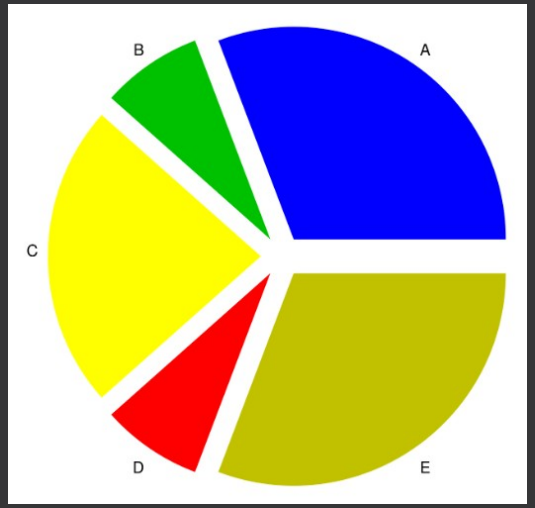
Initialization: OpenBao

Token: The core method for authentication within OpenBao. Tokens can be used directly or auth methods can be used to dynamically generate tokens based on external identities. Without the root token it is not possible to administer OpenBao.

Unseal Key: Unsealing is the process of obtaining the plaintext root key necessary to read the decryption key to decrypt the data, allowing access to the OpenBao service.

©2025 Percona

Initialization: Understanding the key



©2025 Percona

Initialization: generates key, token

```
Active Internet connections (only servers)
Proto Recv-Q Send-Q Local Address           Foreign Address         State       PID/Program name
tcp        0      0 127.0.0.53:53           0.0.0.0:*               LISTEN      211/systemd-resolve
tcp        0      0 0.0.0.0:8200            0.0.0.0:*               LISTEN      1061/bao
tcp        0      0 127.0.0.54:53           0.0.0.0:*               LISTEN      211/systemd-resolve
```

```
export VAULT_ADDR="http://127.0.0.1:8200"
bao operator init -key-shares=1 -key-threshold=1 | grep -E 'Unseal|Token' > bao.ini
```

```
root@openbao:~# export VAULT_ADDR="http://127.0.0.1:8200"
bao operator init -key-shares=1 -key-threshold=1 | grep -E 'Unseal|Token'
Unseal Key 1: LQt0F4m5ZuWtw9awGwiRf5/UNOEb4TqgJk8aP2z/OuE=
Initial Root Token: s.QKH11VesQVHLykl2fursdNjK
```


©2025 Percona

Create Mount Point “percona”:

```
# Recall: bao.ini

UNSEAL_KEY=$(cat bao.ini | grep Unseal | cut -d ':' -f 2)
TOKEN=$(cat bao.ini | grep Token | cut -d ':' -f 2)

export VAULT_ADDR="http://127.0.0.1:8200" \
       VAULT_TOKEN=$TOKEN

bao operator unseal $UNSEAL_KEY
bao secrets enable -path=percona --version=2 kv
bao secrets list
```

```
root@openbao:~# UNSEAL_KEY=$(cat bao.ini | grep Unseal | cut -d ':' -f 2)
TOKEN=$(cat bao.ini | grep Token | cut -d ':' -f 2)

export VAULT_ADDR="http://127.0.0.1:8200" \
       VAULT_TOKEN=$TOKEN
root@openbao:~# bao operator unseal $UNSEAL_KEY
Key             Value
---             -----
Seal Type       shamir
Initialized     true
Sealed          false
Total Shares    1
Threshold       1
Version         2.2.1
Build Date      2025-04-22T20:36:25Z
Storage Type    file
Cluster Name    vault-cluster-ad9dd398
Cluster ID      305690e6-b32d-5ab5-92de-3ecd9d13c85d
HA Enabled      false
```

```
root@openbao:~# bao secrets enable -path=percona --version=2 kv
Success! Enabled the kv secrets engine at: percona/
root@openbao:~# bao secrets list
Path          Type         Accessor              Description
----          ----         --------              -----------
cubbyhole/    cubbyhole    cubbyhole_7107fe00    per-token private secret storage
identity/     identity     identity_fd8b5c90     identity store
percona/      kv           kv_8354cacd           n/a
sys/          system       system_45b47e01       system endpoints used for control, policy and debugging
root@openbao:~#
```


©2025 Percona

Create Provider:

```
export VAULT_TOKEN=$(cat bao.ini | grep Token | cut -d ':' -f 2)

su - postgres <<_eof_
    dropdb --if-exists db01
    createdb db01 --template=tde_template

    psql db01 <<_eof1_
       \set ON ERROR STOP

        select * from pg_tde_add_database_key_provider_vault_v2 (
            'percona01',
            '$VAULT_TOKEN',
            'http://127.0.0.1:8200',
            'percona',
            NULL
        );

        select * from pg_tde_set_key_using_database_key_provider (
            'key01',
            'percona01',
            true
        );

        select * from pg_tde_list_all_database_key_providers();
        select * from pg_tde_key_info();
_eof1_
_eof_
```


©2025 Percona

About CLI bao:

```
root@openbao:~# bao --help
Usage: bao <command> [args]

Common commands:
    read        Read data and retrieves secrets
    write       Write data, configuration, and secrets
    delete      Delete secrets and configuration
    list        List data or secrets
    login       Authenticate locally
    agent       Start an OpenBao agent
    server      Start an OpenBao server
    status      Print seal and HA status
    unwrap      Unwrap a wrapped secret

Other commands:
    audit                Interact with audit devices
    auth                 Interact with auth methods
    debug                Runs the debug command
    kv                   Interact with OpenBao's Key-Value storage
    lease                Interact with leases
    monitor              Stream log messages from an OpenBao server
    namespace            Interact with namespaces
    operator             Perform operator-specific tasks
    patch                Patch data, configuration, and secrets
    path-help            Retrieve API help for paths
    pki                  Interact with OpenBao's PKI Secrets Engine
    plugin               Interact with OpenBao plugins and catalog
    policy               Interact with policies
    print                Prints runtime configurations
    proxy                Start an OpenBao Proxy
    scan                 Scan (recursively list) data or secrets
    secrets              Interact with secrets engines
    ssh                  Initiate an SSH session
    token                Interact with tokens
    transit              Interact with OpenBao's Transit Secrets Engine
    version-history      Prints the version history of the target Vault server
root@openbao:~#
```


©2025 Percona

Configuring Openbao Using TLS:

Working with OpenSSL Certificates

STEPS:

1) Update the OpenBao configuration file, *openbao.hcl*
2) Create the OpenSSL configuration file, *openssl.cnf*
3) Create a new openbao private key, *server.key*
4) Create the certificate request, CSR, *server.csr*
5) Sign the CSR using a RootCA
6) Restart/Unseal OpenBao

©2025 Percona

Update The OpenBao Configuration File, *openbao.hcl*

```
ui = true

storage "file" {
  path = "/opt/openbao/data"
}

# HTTP listener
listener "tcp" {
  address = "0.0.0.0:8200"
  tls_cert_file = "/opt/openbao/tls/server.crt"
  tls_key_file  = "/opt/openbao/tls/server.key"
}
```


©2025 Percona

Create The OpenSSL Configuration File, *openssl.cnf*

```
[req]
default_bits=2048
prompt = no
default_md = sha256
distinguished_name = req_distinguished_name
req_extensions = v3_req
x509_extensions = v3_ca

[req_distinguished_name]
C = US
ST = Washington
L = Seattle
O = Percona
CN = pg1

[v3_ca]
basicConstraints = critical,CA:true

[v3_req]
subjectAltName = @alt_names

[alt_names]
IP.1 = 127.0.0.1
IP.2 = 10.108.134.140
```


©2025 Percona

Create A Key, CSR And Signed Certificate

server.key, server.csr, server.crt

```
# generate new key
openssl genrsa -out server.key 4096

# create CSR
openssl req -new -key server.key -out server.csr -config openssl.cnf

# sign CSR
openssl x509 -req -in server.csr \
        -CA tls.crt -CAkey tls.key \
        -out server.crt -days 500 -sha256 \
        -extfile openssl.cnf -extensions v3_req
```


©2025 Percona

Create The Certificate Request

```
Certificate Request:
    Data:
        Version: 1 (0x0)
        Subject: C = US, ST = Washington, L = Seattle, O = Percona, CN = pg1
        Subject Public Key Info:
            Public Key Algorithm: rsaEncryption
                Public-Key: (4096 bit)
                Modulus:
                    00:bb:33:3e:d9:10:85:e2:cb:da:b8:72:99:27:84:
                    37:25:a2:93:63:b3:18:ef:96:80:15:73:2e:72:ab:
                    50:f5:ec:98:9b:d7:31:c7:0b:08:83:55:49:a0:1d:
                    fa:fb:17:3f:5e:01:22:b4:22:e9:2b:d0:35:c5:37:
                    89:0d:57:15:81:94:8e:21:0c:58:09:11:5f:0d:2b:
                    fd:eb:e2:07:4f:3b:2c:73:fb:c9:af:56:5e:80:a9:
                    6b:3e:1b:21:2d:d3:f4:d8:1d:65:3c:33:a5:69:bc:
                    f9:94:65:b8:d5:49:17:01:cb:ed:14:52:d1:9a:b6:
                    9c:ea:61:57:96:da:fd:ee:f2:98:38:8c:19:df:62:
                    5e:0e:20:a4:e4:33:70:c0:56:65:26:f4:a1:a1:f5:
                    94:77:45:5b:dd:e9:41:53:6f:d3:fb:28:d4:c8:9b:
                    ec:cc:77:07:11:76:3f:5a:4f:7c:11:a0:95:2d:3e:
                    8d:17:d4:6b:ed:08:7d:b0:90:6b:1d:f3:fc:5d:9e:
                    9e:00:f8:80:e8:be:b5:cd:cc:c9:71:33:d3:c0:64:
                    77:52:fb:85:77:f2:33:87:d5:80:45:73:b6:01:ae:
                    26:06:95:98:09:23:02:bd:bd:d6:e5:f1:1b:1f:e5:
                    b4:8c:79:5f:46:0f:e3:45:ec:6c:0d:86:dc:d2:77:
                    c3:ee:5e:02:cf:94:f5:c8:90:0d:d8:fd:d8:0f:cd:
                    91:d5:56:87:c5:a4:37:9b:23:b6:d0:f4:46:89:a2:
                    06:0e:55:ea:53:55:97:be:d5:36:b6:8d:f9:85:a8:
                    9c:19:80:84:28:66:8c:7f:df:49:78:50:52:60:c5:
                    70:eb:a5:1f:60:7a:0d:b0:21:a5:30:98:ff:f6:01:
                    52:d0:ce:b9:a7:10:8f:2a:bb:6d:10:fa:e8:56:33:
                    19:50:e2:e0:0f:47:fb:10:86:91:b5:f1:17:68:70:
                    3d:c5:c7:76:df:85:2e:4c:76:97:4f:4a:f3:b1:3a:
                    6c:4b:71:86:9a:53:84:1d:fe:a6:2b:fd:a3:8a:4a:
                    72:40:49:7f:4a:0e:70:a8:26:94:ff:4b:f3:44:77:
                    ad:9e:fd:e7:d2:dd:24:8c:60:11:53:b0:5a:47:ab:
                    77:4a:64:55:22:02:09:7e:c1:bd:ac:fd:76:e0:91:
                    77:db:76:53:c4:1d:4b:bb:16:53:bb:f1:fd:af:3e:
                    70:90:c2:ed:3a:47:7c:bd:52:b7:0a:f3:6b:61:76:
                    97:c1:6d:00:cc:77:c7:b8:c4:21:3f:2c:71:54:89:
                    73:e4:d2:87:40:9a:f5:24:24:30:fd:67:ac:79:86:
                    38:62:90:88:97:92:69:3e:c5:6e:fd:4a:f6:6f:4f:
                    a9:6c:fb
                Exponent: 65537 (0x10001)
        Attributes:
            Requested Extensions:
                X509v3 Subject Alternative Name:
                    IP Address:127.0.0.1
    Signature Algorithm: sha256WithRSAEncryption
    Signature Value:
        70:4b:77:a7:4d:c8:83:56:90:a9:89:d2:4c:e8:dd:20:31:0f:
        40:a2:ea:60:dc:82:d5:b6:0c:a0:39:e9:7a:3c:4b:c8:e7:83:
        99:cd:96:ae:9f:82:70:b4:be:5d:67:c5:a1:c6:0b:e4:c1:c5:
        b6:92:68:a0:cf:5c:b9:02:88:f9:a3:79:eb:1d:2b:56:d6:94:
        db:fc:21:fc:a4:88:77:fa:3a:cd:f1:3f:c9:12:98:97:34:52:
```

```
openssl req -in /opt/openbao/tls/server.csr -text -noout | less
```

```
Attributes:
    Requested Extensions:
        X509v3 Subject Alternative Name:
            IP Address:127.0.0.1
```


©2025 Percona

Sign The CSR Using A RootCA

```
openssl x509 -in /opt/openbao/tls/tls.crt -text -noout | less
```

```
Certificate:
    Data:
        Version: 3 (0x2)
        Serial Number:
            6e:20:93:5d:57:1f:fa:3e:3c:ee:e3:be:7e:18:67:4c:77:ed:52:7d
        Signature Algorithm: sha256WithRSAEncryption
        Issuer: O = OpenBao, CN = OpenBao
        Validity
            Not Before: Jun 23 22:39:35 2025 GMT
            Not After : Jun 22 22:39:35 2028 GMT
        Subject: O = OpenBao, CN = OpenBao
        Subject Public Key Info:
            Public Key Algorithm: rsaEncryption
                Public-Key: (4096 bit)
                Modulus:
                    00:cd:17:5a:90:14:37:32:32:66:2a:4f:6c:b8:aa:
                    04:5d:96:1d:1b:c2:72:9c:80:1f:66:6a:59:56:0a:
                    fb:1f:13:6b:2a:cc:22:58:c4:27:3c:49:d9:3a:4b:
                    8f:bc:6b:42:e1:a1:cd:d6:7e:5b:36:52:7c:c2:98:
                    34:14:a7:6c:19:e3:b3:9e:1d:43:db:3a:c1:3b:17:
                    5b:46:52:05:41:73:83:85:26:8c:4a:0b:c5:06:5d:
                    82:14:28:6b:c0:aa:2a:87:5e:0b:f9:36:91:e6:00:
                    49:26:c5:e3:a0:da:dc:57:78:66:00:8c:b0:d2:e4:
                    73:f1:b2:83:a6:59:d0:c8:fa:76:04:29:e4:00:73:
                    9c:5b:a4:ae:5a:77:aa:4f:25:6f:47:3c:93:f2:96:
                    76:07:89:d4:5d:df:06:9a:23:f9:f4:32:45:b2:1b:
                    76:31:1a:9a:f3:ad:e3:de:5a:e2:d7:f3:34:2a:1f:
                    b5:3d:ac:9f:e9:f2:ae:0c:af:b2:95:22:8a:ed:94:
                    a5:29:0a:3e:cf:40:e6:b0:1d:74:4c:49:06:b8:b1:
                    95:7a:ca:dc:26:de:f1:4c:f9:5f:ea:28:76:d8:eb:
                    af:21:7c:83:03:f1:b5:6d:b4:ad:a7:34:dd:c3:f6:
                    c5:f0:cf:64:1b:a7:47:ae:c2:ce:a0:27:af:8d:e3:
                    c4:56:f5:e1:a7:29:7f:1c:a0:c8:fa:5e:a6:6d:e3:
                    b6:24:c5:da:67:78:5f:0c:dc:ba:70:ff:e0:fb:5b:
                    5e:6e:bf:75:31:b7:05:18:0f:24:2c:ed:00:b9:a9:
                    b4:36:2e:26:60:b6:47:5a:74:9b:51:8f:23:73:00:
                    b0:7c:d8:64:77:81:25:ff:c7:b5:16:f2:e6:90:aa:
                    d5:83:73:84:b1:c2:cd:1c:57:8c:c8:ce:7c:7f:36:
                    78:e2:65:b1:47:ec:01:47:72:bc:8a:63:51:9f:c2:
                    20:ed:4b:fB:bd:aa:8a:df:85:df:70:53:40:10:9e:
                    f0:26:f9:c8:7b:72:b9:d5:32:36:43:31:2c:c3:0d:
                    f6:ad:ef:f6:c3:2f:12:fa:29:5c:cd:e3:d0:1f:0e:
                    21:4c:55:7b:a3:7d:87:d8:Bd:85:2e:3d:bb:e1:a0:
                    e6:9e:f0:27:ad:bd:18:7e:cd:8d:de:1e:18:54:19:
                    93:d4:58:bc:83:40:e2:4b:62:68:f7:2f:8c:fa:72:
                    69:13:f6:2d:ed:e5:82:4f:f4:8b:cb:01:48:c7:63:
                    c8:26:c0:fb:d3:e1:1c:f5:b5:98:0d:67:70:5a:15:
                    82:c9:60:62:21:d0:b1:73:b9:12:4b:68:48:31:f2:
                    c2:01:58:rr:B4:ba:4f:97:35:5a:a9:d4:2e:56:0f:
                    da:34:61
                Exponent: 65537 (0x10001)
        X509v3 extensions:
            X509v3 Subject Key Identifier:
                35:31:2F:CA:B8:50:9C:8A:13:13:0A:A7:34:24:20:17:58:5E:E4:E4
            X509v3 Authority Key Identifier:
                35:31:2F:CA:B8:50:9C:8A:13:13:0A:A7:34:24:20:17:58:5E:E4:E4
            X509v3 Basic Constraints: critical
                CA:TRUE
    Signature Algorithm: sha256WithRSAEncryption
    Signature Value:
        c3:00:73:ec:6a:36:0e:3a:b2:a5:bb:6a:f3:49:9f:2f:bf:cd:
        de:8c:09:61:9b:cc:fe:e0:f1:fe:15:26:33:20:49:41:da:d2:
        a2:69:6e:85:ee:cd:2a:9c:ae:d7:96:3f:b8:48:62:7c:47:fc:
        10:89:82:6f:11:c7:3d:1c:83:0b:06:f8:63:31:da:b5:f8:25:
```

```
X509v3 extensions:
    X509v3 Subject Key Identifier:
        35:31:2F:CA:B8:50:9C:8A:13:13:0A:A7:34:24:20:17:58:5E:E4:E4
    X509v3 Authority Key Identifier:
        35:31:2F:CA:B8:50:9C:8A:13:13:0A:A7:34:24:20:17:58:5E:E4:E4
    X509v3 Basic Constraints: critical
        CA:TRUE
```


©2025 Percona

Restart/Unseal The OpenBao Service

```
cd $HOME
export UNSEAL_KEY=$(cat bao.ini | grep Unseal | cut -d ':' -f 2)
       TOKEN=$(cat bao.ini | grep Token | cut -d ':' -f 2) \
       VAULT_ADDR="https://127.0.0.1:8200"

echo "$TOKEN" > $HOME/.vault_toke

bao operator unseal -ca-cert /opt/openbao/tls/tls.crt $UNSEAL_KEY
```

```
Key             Value
---             -----
Seal Type       shamir
Initialized     true
Sealed          false
Total Shares    1
Threshold       1
Version         2.3.0-beta20250528+hsm
Build Date      2025-05-28T21:53:09Z
Storage Type    file
Cluster Name    vault-cluster-ebc6d3bb
Cluster ID      e45aad85-b04f-19cc-ab0a-ef6887309ff6
HA Enabled      false
```


©2025 Percona

Create New Provider

STEPS:

1) Add the new provider
2) Set a new key
3) Delete old provider
4) Restart PostgreSQL
5) Validate system

©2025 Percona

Update The pg_tde Vault Provider

Recall

```
echo "$TOKEN" > $HOME/.vault_token
```

```
# ensure tls.cert and .vault_token are present in current directory

export BAO_CACERT=tls.crt \
       VAULT_ADDR="https://127.0.0.1:8200" \
       VAULT_TOKEN=$(cat .vault_token)

bao kv list -mount=percona
```

```
-- add the new provider
select * from pg_tde_add_database_key_provider_vault_v2 (
    'percona02',
    'https://127.0.0.1:8200',
    'percona',
    '/var/lib/postgresql/.vault_token',
    '/var/lib/postgresql/tls.crt'
);

select * from pg_tde_create_key_using_database_key_provider(
    'key02',
    'percona02'
);

select pg_tde_set_key_using_database_key_provider (
    'key02',
    'percona02'
);
```


©2025 Percona

Validate: Restart PostgreSQL

```
systemctl restart postgresql@17-main
```


©2025 Percona

Validate System

```
/* RECALL TABLE T1
                                    Table "public.t1"
  Column  |           Type           | Collation | Nullable |            Default
----------+--------------------------+-----------+----------+--------------------------------
 id       | integer                  |           | not null | nextval('t1_id_seq'::regclass)
 comments | text                     |           |          | 'hello world, t1'::text
 t_stamp  | timestamp with time zone |           |          | now()
Indexes:
    "t1_pkey" PRIMARY KEY, btree (id)

*/

insert into t1 values (default, default, default);
table t1;

select * from pg_tde_list_all_database_key_providers();
select * from pg_tde_key_info();
```


©2025 Percona

Validate System, cont'd

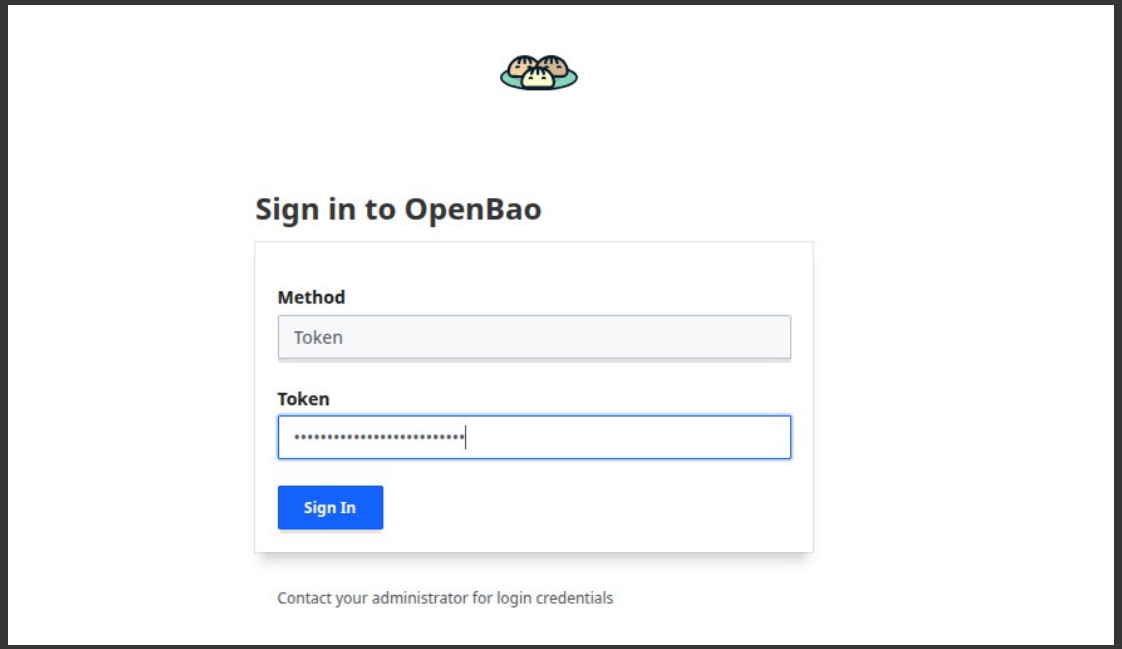
```
 pg_tde_add_database_key_provider_vault_v2
-------------------------------------------
                                         2
(1 row)

 pg_tde_set_key_using_database_key_provider
--------------------------------------------

(1 row)

 pg_tde_delete_database_key_provider
-------------------------------------

(1 row)
```

```
INSERT 0 1
 id |    comments      |            t_stamp
----+------------------+-------------------------------
  1 | hello world, t1 | 2025-06-24 21:06:20.152441+00
  2 | hello world, t1 | 2025-06-24 21:06:20.157227+00
  3 | hello world, t1 | 2025-06-24 21:06:20.161234+00
  4 | hello world, t1 | 2025-06-24 21:06:53.990779+00
(4 rows)
```

```
 id | provider_name | provider_type |                                                                                                                                                                                                                                                                                                                                                options
----+---------------+---------------+-------------------------------------------
```

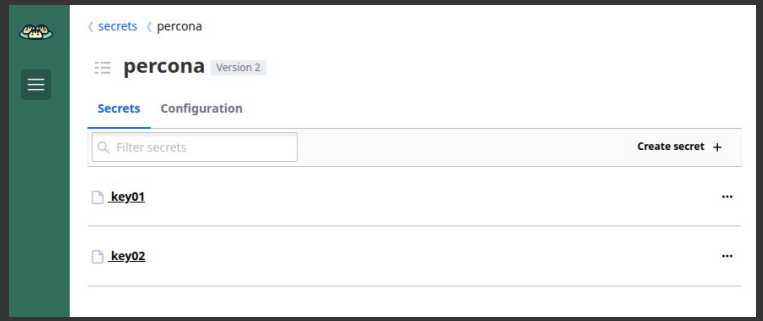
Validate System, ui

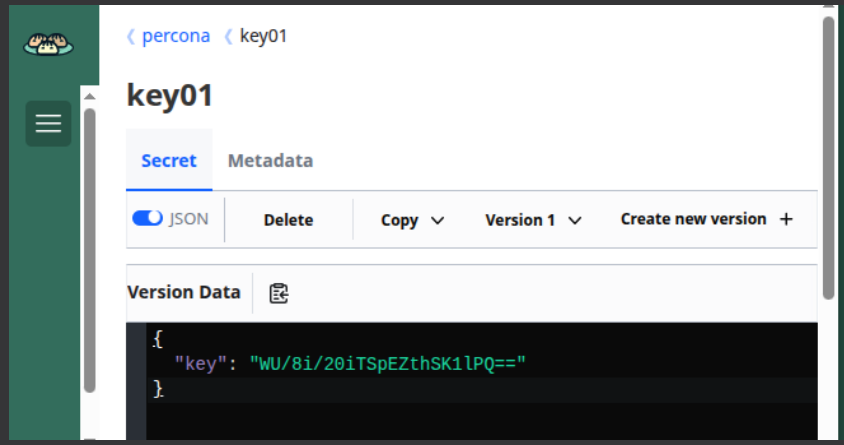
https://###.###.###.###:8200/ui

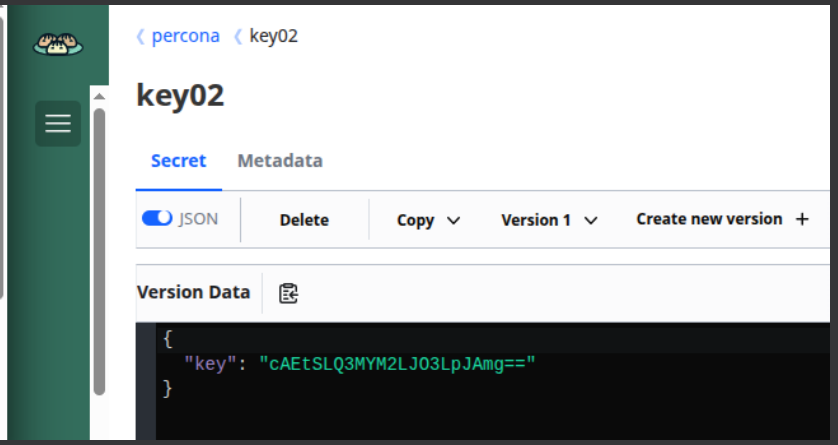


©2025 Percona

Validate System, ui

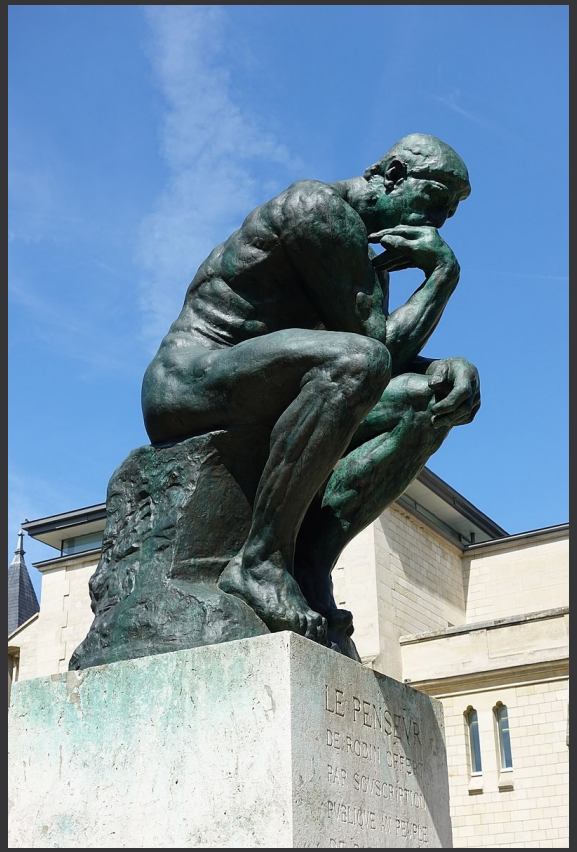






©2025 Percona

Review …



©2025 Percona

Functions

pg_tde_add_database_key_provider	pg_tde_change_global_key_provider_kmip
pg_tde_add_database_key_provider_file	pg_tde_change_global_key_provider_vault_v2
pg_tde_add_database_key_provider_file	pg_tde_change_global_key_provider_vault_v2
pg_tde_add_database_key_provider_kmip	pg_tde_ddl_command_end_capture
pg_tde_add_database_key_provider_kmip	pg_tde_ddl_command_start_capture
pg_tde_add_database_key_provider_vault_v2	pg_tde_default_key_info
pg_tde_add_database_key_provider_vault_v2	pg_tde_delete_database_key_provider
pg_tde_add_global_key_provider	pg_tde_delete_global_key_provider
pg_tde_add_global_key_provider_file	pg_tde_grant_database_key_management_to_role
pg_tde_add_global_key_provider_file	pg_tde_grant_key_viewer_to_role
pg_tde_add_global_key_provider_kmip	pg_tde_is_encrypted
pg_tde_add_global_key_provider_kmip	pg_tde_key_info
pg_tde_add_global_key_provider_vault_v2	pg_tde_list_all_database_key_providers
pg_tde_add_global_key_provider_vault_v2	pg_tde_list_all_global_key_providers
pg_tde_change_database_key_provider	pg_tde_revoke_database_key_management_from_role
pg_tde_change_database_key_provider_file	pg_tde_revoke_key_viewer_from_role
pg_tde_change_database_key_provider_file	pg_tde_server_key_info
pg_tde_change_database_key_provider_kmip	pg_tde_set_default_key_using_global_key_provider
pg_tde_change_database_key_provider_kmip	pg_tde_set_key_using_database_key_provider
pg_tde_change_database_key_provider_vault_v2	pg_tde_set_key_using_global_key_provider
pg_tde_change_database_key_provider_vault_v2	pg_tde_set_server_key_using_global_key_provider
pg_tde_change_global_key_provider	pg_tde_verify_default_key
pg_tde_change_global_key_provider_file	pg_tde_verify_key
pg_tde_change_global_key_provider_file	pg_tde_verify_server_key
pg_tde_change_global_key_provider_kmip	pg_tde_version
	pg_tdeam_handler

©2025 Percona

Runtime Parameters

```
db01=# select name,setting from pg_settings where name ~ '^pg_tde';
              name                | setting
----------------------------------+---------
 pg_tde.enforce_encryption        | off
 pg_tde.inherit_global_providers  | on
 pg_tde.wal_encrypt               | on
(3 rows)
```


©2025 Percona

Questions?

©2025 Percona

Caveat

©2025 Percona

Thank You :-)

©2025 Percona